Sudoku Solver GenAI Comparison

1. Objective

I need to create a sudoku solver that recursively solves puzzles that are accessed by input files and print the solutions to the terminal.

GenAI Access and Shared Prompt

How I accessed the programs: I used Gemini through its web interface and CoPilot through its desktop application

Prompt used for both GenAIs:

> generate C++ code using objects, not functions, to implement the following requirements: • Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub- grids, or blocks. • A typical puzzle is shown here (image credit: Wikimedia Commons): o • Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints: o Each digit from 1 to 9 must appear exactly once in each row. o Each digit from 1 to 9 must appear exactly once in each column. o Each digit from 1 to 9 must appear exactly once in each block. • Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve. • The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that. • It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it’s possible to gradually fill in the entire grid. • That is not the approach we’re going to use. Instead, we’re going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by “slower to run,” we mean it'll finish in a few tenths of a second rather than a few milliseconds.) • Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier). • This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities. • You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square. • Write a recursive object-oriented program to find the solution to the five puzzle files. • The output for each

> solution, should include: o The puzzle file name (e.g., puzzle1.txt) o The puzzle from the file printed out. o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out. • The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out. • To help you debug your program, solution1.txt is a solution to puzzle1.txt.

---

3. GenAI #1

GenAI Used: Gemini

Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

// Represents the Sudoku Board and encapsulates the backtracking solver
class SudokuBoard {
private:
    char grid[9][9];

    // Helper to check if placing 'num' at grid[row][col] is valid
    bool isValid(int row, int col, char num) const {
        for (int i = 0; i < 9; i++) {
            // Check row and column
            if (grid[row][i] == num) return false;
            if (grid[i][col] == num) return false;
        }

        // Check 3x3 sub-grid / block
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int r = 0; r < 3; r++) {
            for (int c = 0; c < 3; c++) {
                if (grid[startRow + r][startCol + c] == num) return false;
            }
        }

        return true;
    }

    // Recursive backtracking method to find all solutions
    bool solveRecursive(int row, int col, bool findMultiple, int& solutionsCount)
{
        // Reached end of row/grid, puzzle solved!
        if (row == 9) {
            solutionsCount++;
            std::cout << "--- Solution " << solutionsCount << " ---\n";
            print();
```

```
            std::cout << "\n";
            // If we want multiple solutions, return false here to force
backtracking and find others.
            // If we only wanted the first, we would return true.
            return !findMultiple;
        }

        // Move to next row if we finish the current column
        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        // If cell is already filled, move to the next
        if (grid[row][col] != '_') {
            return solveRecursive(nextRow, nextCol, findMultiple, solutionsCount);
        }

        bool found = false;
        // Try digits '1' through '9'
        for (char num = '1'; num <= '9'; num++) {
            if (isValid(row, col, num)) {
                grid[row][col] = num; // Place candidate

                if (solveRecursive(nextRow, nextCol, findMultiple,
solutionsCount)) {
                    found = true;
                    if (!findMultiple) break;
                }

                grid[row][col] = '_'; // Backtrack
            }
        }

        return found;
    }

public:
    // Constructor initializes an empty board
    SudokuBoard() {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                grid[i][j] = '_';
            }
        }
    }

    // Load puzzle from file
    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
```

```
                file >> grid[i][j];
            }
        }
        return true;
    }

    // Print the puzzle grid nicely
    void print() const {
        for (int i = 0; i < 9; i++) {
            if (i > 0 && i % 3 == 0) {
                std::cout << "------+-------+------\n";
            }
            for (int j = 0; j < 9; j++) {
                if (j > 0 && j % 3 == 0) {
                    std::cout << "| ";
                }
                std::cout << grid[j] << " "; // Wait, grid[i][j] formatting:
            }
            // Correction for row/col printing below:
            // Let's rewrite print correctly:
        }
    }

    // Corrected print function
    void display() const {
        for (int i = 0; i < 9; i++) {
            if (i > 0 && i % 3 == 0) {
                std::cout << "------+-------+------\n";
            }
            for (int j = 0; j < 9; j++) {
                if (j > 0 && j % 3 == 0) {
                    std::cout << "| ";
                }
                std::cout << grid[i][j] << " ";
            }
            std::cout << "\n";
        }
    }

    // Solve interface method
    void solve(const std::string& filename, bool findMultiple = true) {
        std::cout << "========================================\n";
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "========================================\n";

        if (!loadFromFile(filename)) {
            std::cout << "Error: Could not open file " << filename << "\n\n";
            return;
        }

        std::cout << "Initial Puzzle:\n";
        display();
        std::cout << "\nResolving...\n\n";
```

```
        int solutionsCount = 0;
        solveRecursive(0, 0, findMultiple, solutionsCount);

        if (solutionsCount == 0) {
            std::cout << "No solution found\n\n";
        } else {
            std::cout << "Total solutions found: " << solutionsCount << "\n\n";
        }
    }
};

int main() {
    // Process puzzle files puzzle1.txt through puzzle5.txt
    for (int i = 1; i <= 5; i++) {
        std::string filename = "puzzle" + std::to_String(i) + ".txt"; // wait,
standard string is std::to_string
        // Using lowercase 's' for std::to_string:
    }

    // Clean loop implementation:
    for (int i = 1; i <= 5; i++) {
        std::string filename = "puzzle" + std::to_string(i) + ".txt";
        SudokuBoard board;
        board.solve(filename, true); // Set to 'true' to find all solutions if
multiples exist
    }

    return 0;
}
```

4. GenAI #2

GenAI Used: CoPilot

Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuGrid {
private:
    int grid[9][9];          // 0 means empty
    std::string sourceName;  // file name

public:
    SudokuGrid() : sourceName("") {
        for (int r = 0; r < 9; ++r)
            for (int c = 0; c < 9; ++c)
```

```
                grid[r][c] = 0;
    }

    void setSourceName(const std::string &name) {
        sourceName = name;
    }

    const std::string &getSourceName() const {
        return sourceName;
    }

    bool loadFromFile(const std::string &filename) {
        setSourceName(filename);
        std::ifstream in(filename);
        if (!in) return false;

        std::string line;
        int row = 0;
        while (std::getline(in, line) && row < 9) {
            int col = 0;
            for (char ch : line) {
                if (col >= 9) break;
                if (ch == '_' || ch == '.' || ch == '0') {
                    grid[row][col] = 0;
                    ++col;
                } else if (ch >= '1' && ch <= '9') {
                    grid[row][col] = ch - '0';
                    ++col;
                }
            }
            if (col == 9) ++row;
        }
        return row == 9;
    }

    void print(std::ostream &out) const {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                if (grid[r][c] == 0) out << "_";
                else out << grid[r][c];
            }
            out << "\n";
        }
    }

    int get(int r, int c) const {
        return grid[r][c];
    }

    void set(int r, int c, int val) {
        grid[r][c] = val;
    }

    bool isRowValid(int r, int val) const {
```

```
        for (int c = 0; c < 9; ++c)
            if (grid[r][c] == val) return false;
        return true;
    }

    bool isColValid(int c, int val) const {
        for (int r = 0; r < 9; ++r)
            if (grid[r][c] == val) return false;
        return true;
    }

    bool isBlockValid(int r, int c, int val) const {
        int br = (r / 3) * 3;
        int bc = (c / 3) * 3;
        for (int rr = br; rr < br + 3; ++rr)
            for (int cc = bc; cc < bc + 3; ++cc)
                if (grid[rr][cc] == val) return false;
        return true;
    }

    bool isCellEmpty(int r, int c) const {
        return grid[r][c] == 0;
    }

    bool findEmptyCell(int &r, int &c) const {
        for (int i = 0; i < 9; ++i)
            for (int j = 0; j < 9; ++j)
                if (grid[i][j] == 0) {
                    r = i;
                    c = j;
                    return true;
                }
        return false;
    }

    bool isComplete() const {
        for (int r = 0; r < 9; ++r)
            for (int c = 0; c < 9; ++c)
                if (grid[r][c] == 0) return false;
        return true;
    }
};

class SudokuSolver {
private:
    SudokuGrid initial;
    std::vector<SudokuGrid> solutions;

    bool canPlace(const SudokuGrid &g, int r, int c, int val) const {
        return g.isRowValid(r, val) &&
               g.isColValid(c, val) &&
               g.isBlockValid(r, c, val);
    }
```

```
    void search(SudokuGrid &current) {
        int r, c;
        if (!current.findEmptyCell(r, c)) {
            if (current.isComplete()) {
                solutions.push_back(current);
            }
            return;
        }

        bool anyCandidate = false;
        for (int val = 1; val <= 9; ++val) {
            if (canPlace(current, r, c, val)) {
                anyCandidate = true;
                current.set(r, c, val);
                search(current);
                current.set(r, c, 0); // backtrack
            }
        }

        // If no candidate, this path is dead; just return.
        if (!anyCandidate) return;
    }

public:
    SudokuSolver(const SudokuGrid &g) : initial(g) {}

    void solve() {
        solutions.clear();
        SudokuGrid working = initial;
        search(working);
    }

    const std::vector<SudokuGrid> &getSolutions() const {
        return solutions;
    }

    void printReport(std::ostream &out) const {
        out << "Puzzle file: " << initial.getSourceName() << "\n\n";
        out << "Original puzzle:\n";
        initial.print(out);
        out << "\n";

        if (solutions.empty()) {
            out << "No solution found\n\n";
        } else {
            int idx = 1;
            for (const auto &sol : solutions) {
                out << "Solution " << idx++ << ":\n";
                sol.print(out);
                out << "\n";
            }
        }
    }
};
```

```
int main() {
    std::vector<std::string> files = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const auto &fname : files) {
        SudokuGrid grid;
        if (!grid.loadFromFile(fname)) {
            std::cout << "Puzzle file: " << fname << "\n";
            std::cout << "Failed to load puzzle.\n\n";
            continue;
        }

        SudokuSolver solver(grid);
        solver.solve();
        solver.printReport(std::cout);
    }

    return 0;
}
```

Correctness

- Gemini's solver follows the core Sudoku rules by checking the target row, column, and 3x3 block before placing a candidate, so it is logically correct for valid puzzles.
- The recursive backtracking strategy is a standard depth-first search and will eventually explore all valid fillings until it finds a complete solution or proves none exist.
- CoPilot's design is also logically sound because it centralizes validity checks in `isRowValid`, `isColValid`, and `isBlockValid` and then recursively searches for empty cells.
- Both programs can print every valid solution, which matches the assignment requirement for puzzles that may have multiple solutions.
- Gemini has a reliability issue because the final code contains a partially corrected `print` function and a `display` function; the unused or inconsistent printing logic makes the implementation less dependable.
- CoPilot is more consistent overall because it keeps input parsing, validation, solving, and reporting in distinct responsibilities, making correctness easier to verify.

Execution Time

- Both programs use brute-force backtracking, so the worst-case running time is exponential in the number of empty cells: roughly O(9^k) in the worst case where k cells are blank.
- In practice, both reduce the search space heavily because each candidate is checked against the same row, column, and block before it is tried.
- Gemini's search explores all legal candidates for each empty cell and therefore can take longer when a puzzle has many potential branches.

- CoPilot's design is slightly more structured, but it still performs the same style of recursive search and therefore has comparable asymptotic performance.
- For the five assignment puzzles, both implementations should finish quickly enough because the puzzle is fixed-size and the search is heavily pruned by Sudoku constraints.

Space Complexity

- Each board uses a fixed 9x9 grid, so the storage for the actual puzzle is O(81) = O(1) for a fixed-size Sudoku board.
- Each recursive search path can reach a depth of at most 81 cells, so the recursion stack is O(81) = O(1) for this problem size.
- Gemini keeps only the current board state in memory, so its extra memory overhead is small unless the program stores many printed solutions.
- CoPilot stores all discovered solutions in a `std::vector<SudokuGrid>`, so its space usage grows to O(S * 81), where S is the number of solutions found.
- This means CoPilot uses more memory than Gemini when the assignment includes puzzles with many possible solutions, but the added storage makes its reporting more complete and easier to inspect.

Maintainability

- Gemini's code is harder to maintain because it combines solver logic, display logic, and file iteration in the same class without a clear separation of responsibilities.
- The code also contains duplicated or commented-out printing methods, which creates confusion and makes future debugging more difficult.
- CoPilot is easier to maintain because it separates the `SudokuGrid` data structure from the `SudokuSolver` logic and provides clearer method names like `findEmptyCell`, `canPlace`, and `printReport`.
- CoPilot's code is also easier to extend; for example, a future version could add heuristics, candidate lists, or a limit on the number of solutions without rewriting the whole design.
- Gemini is still understandable at a basic level, but it needs more cleanup and validation to be considered robust or production-quality.

5. Comparison and Selection

- Both GenAI programs satisfy the basic Sudoku requirements and use a recursive backtracking approach, so both are functionally valid starting points.
- The CoPilot version is the better choice because it has cleaner class separation, clearer validation logic, and a more straightforward output structure.
- The Gemini version demonstrates the correct idea but contains rough edges such as inconsistent print functions and weaker organization, which makes it less reliable as a final submission.
- CoPilot also handles file input and reporting in a more polished way, which improves both clarity and maintainability.
- For these reasons, the CoPilot-generated solution is the better selection for the final implementation and for any future modifications.

6. Improvements Identified for the Selected Program

- Add validation to ensure each input file contains exactly nine rows with nine valid symbols before solving.
- Add an optional parameter to stop after the first solution when only one answer is needed, which improves runtime for non-all-solutions cases.
- Avoid storing every solution in memory when only a count is required; instead, a counter could be used to save memory.
- Improve output formatting so the printed puzzle and solution match the assignment requirements exactly, including puzzle names and separators.

Final Remarks: The main ways to save time and space complexity happen at the cost of functionality, and for this reason I made the script prompt the user for one of 3 options: Find the first solution to each puzzle, find the number of solutions to each puzzle, or find and print all solutions to all puzzles. Finding only the first solution cuts down runtime, and finding the number of solutions improves space use.

7. Use Case Diagram

DISCLAIMER: I did think of the general structure but the image itself is generated by ai. I gave copilot the overall layout of the program and the components such as the actors, but copilot generated the actual PNG.

Use case:

Solving Sudoku Puzzle

description of use case:

any user who has an unsolved sudoku puzzle and wants it to be solved. They just put it in the desired format and run the `.exe` to quickly find the solution. The file itself is another actor, because the program needs to interact with the file, so there are two actors in total.

flow description

User puts puzzle in text file in desired format -> User starts program -> system loads puzzle, if correct, moves to next step -> propmts user for options -> prints desired results to terminal -> terminate

Detailed use case

Precondition: puzzles exist and are accessible, are named puzzle1.txt etc., and are correctly formatted
Postconditions: For every file, the system must either load it or throw and error

Main scenario:

1. User launches program, system finds puzzle(s) and loads them.
2. System iterates over each file name
3. for each, it reads the 9 lines, parses the input, and creates the grid
4. checks to make sure read is successful
5. calls solver to use backtrack recursion to solve puzzle
6. the solver checks if placement for each number is valid in the row, column, and block

7. if yes, it places number and moves to the next. if no, it back tracks
8. prints solutions according to user input to the terminal